



Agenda



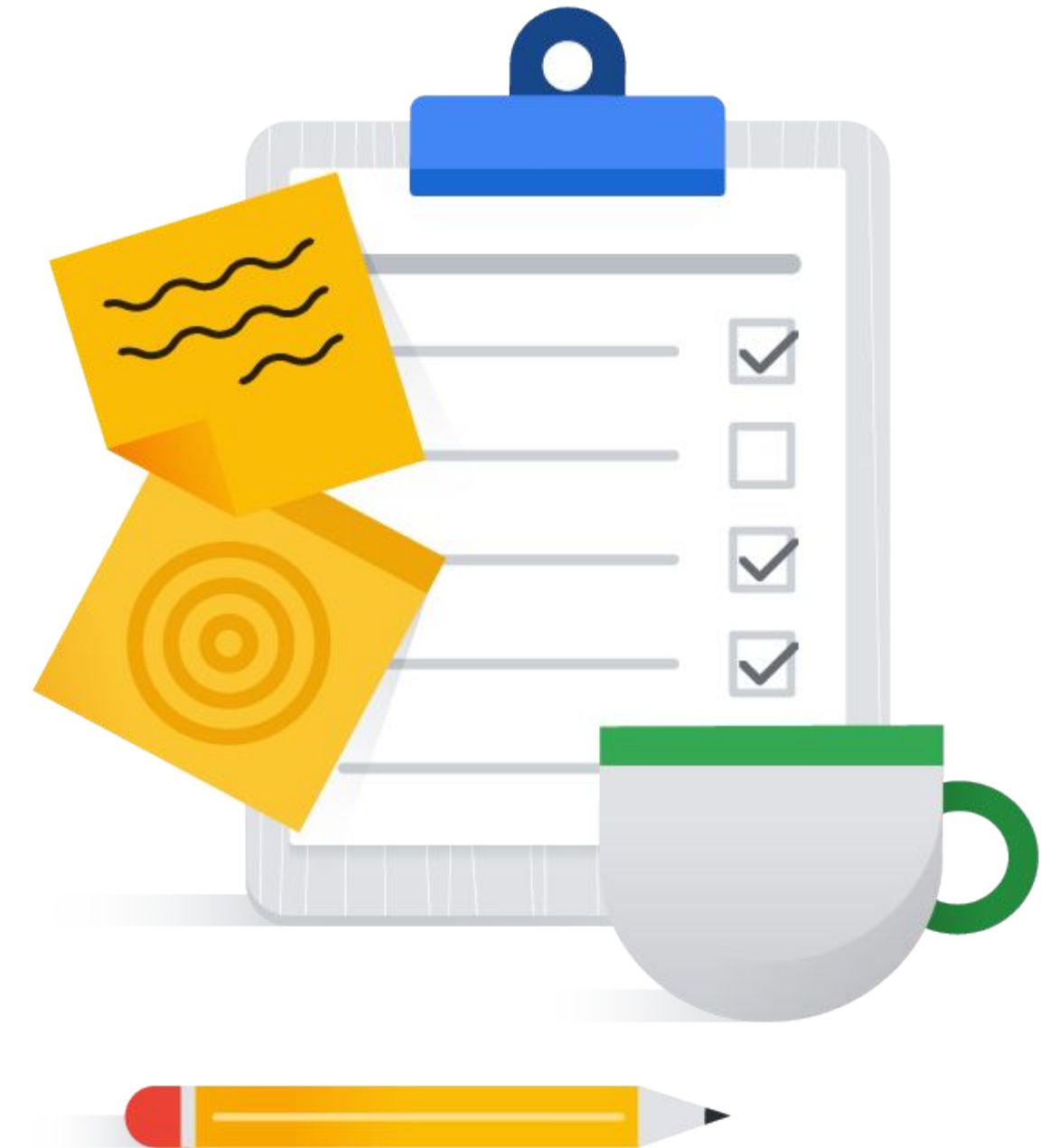
- 01 Introducing Agent Development Kit
- 02 Develop agents with ADK
- 03 Build multi-agent systems with ADK
- 04 Deploy ADK agents to Agent Engine
- 05 Evaluate ADK agent systems



Develop agents with ADK

Module objectives

01	Identify key concepts of ADK.
02	Explain ADK SDK components and functions.
03	Configure an ADK environment.
04	Understand and apply ADK project structure.
05	Describe various methods to interact with agents.
06	Use ADK callbacks to modify agent behavior.



ADK key concepts and SDK

ADK key concepts

Agent Development Kit

ADK key concepts

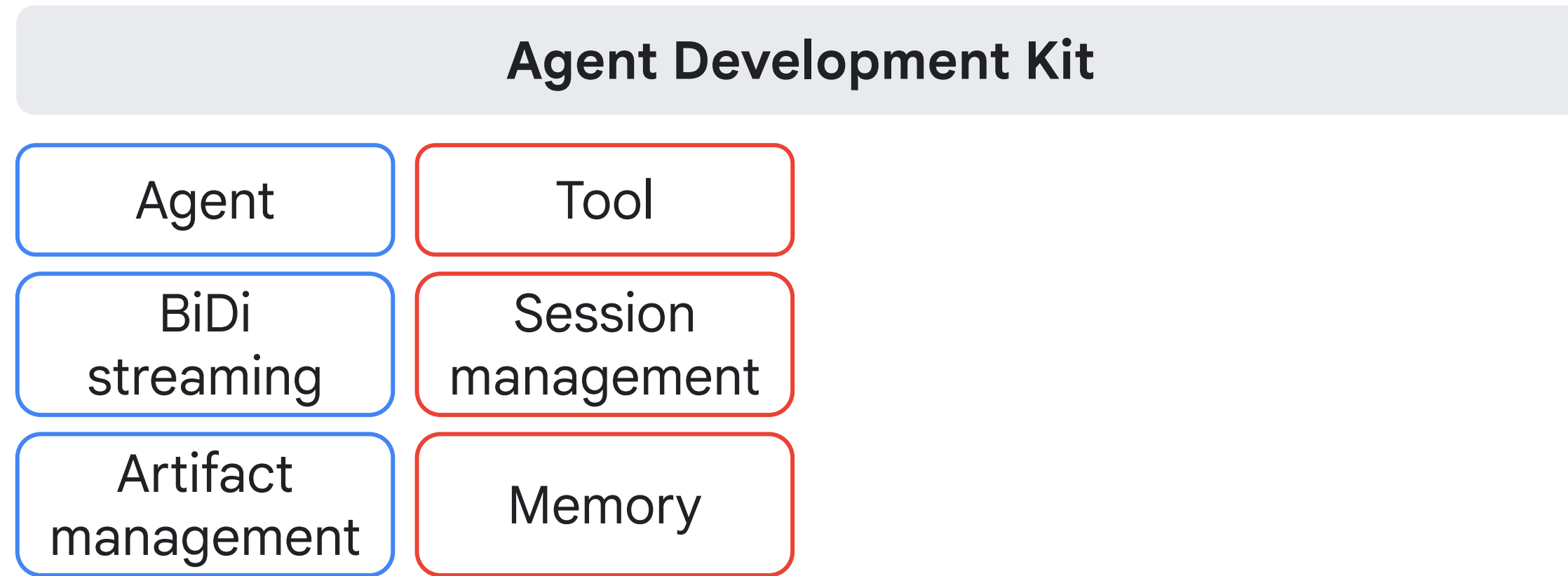
Agent Development Kit

Agent

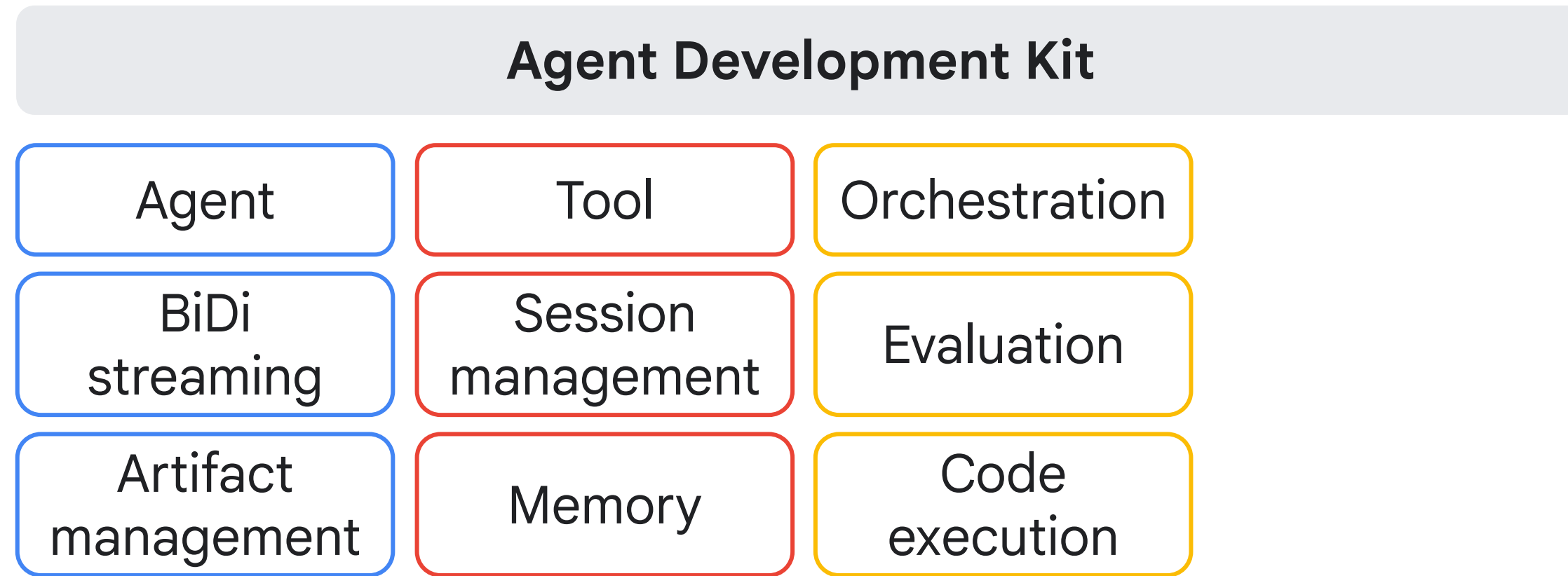
BiDi
streaming

Artifact
management

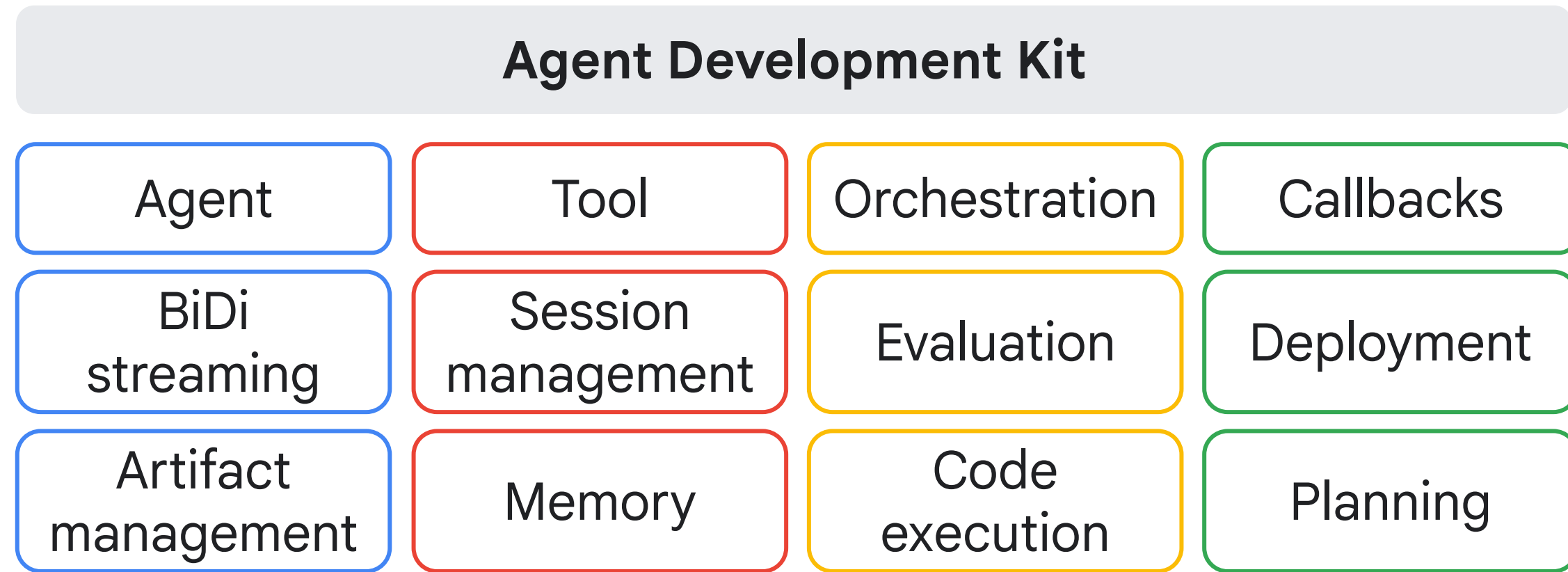
ADK key concepts



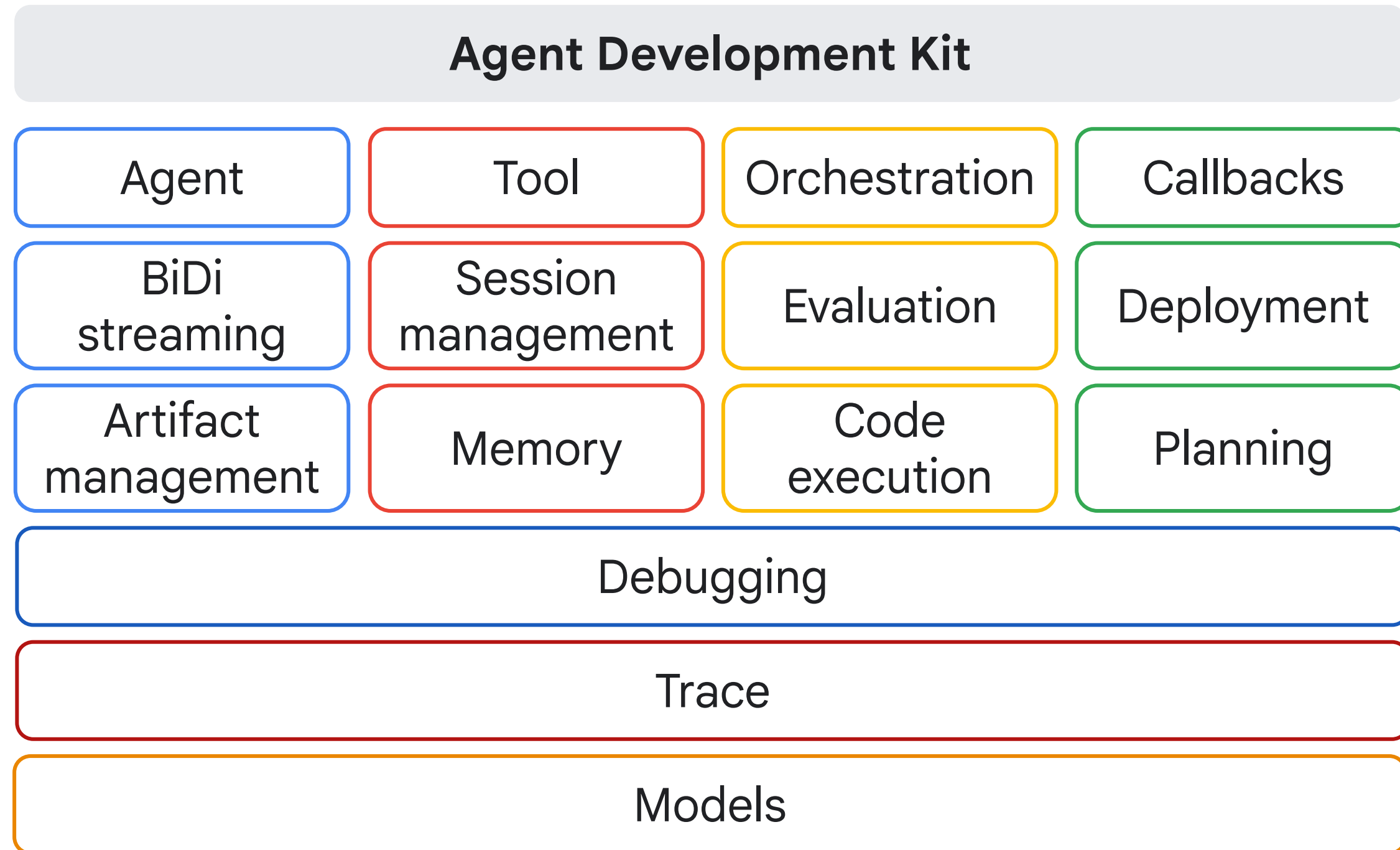
ADK key concepts



ADK key concepts

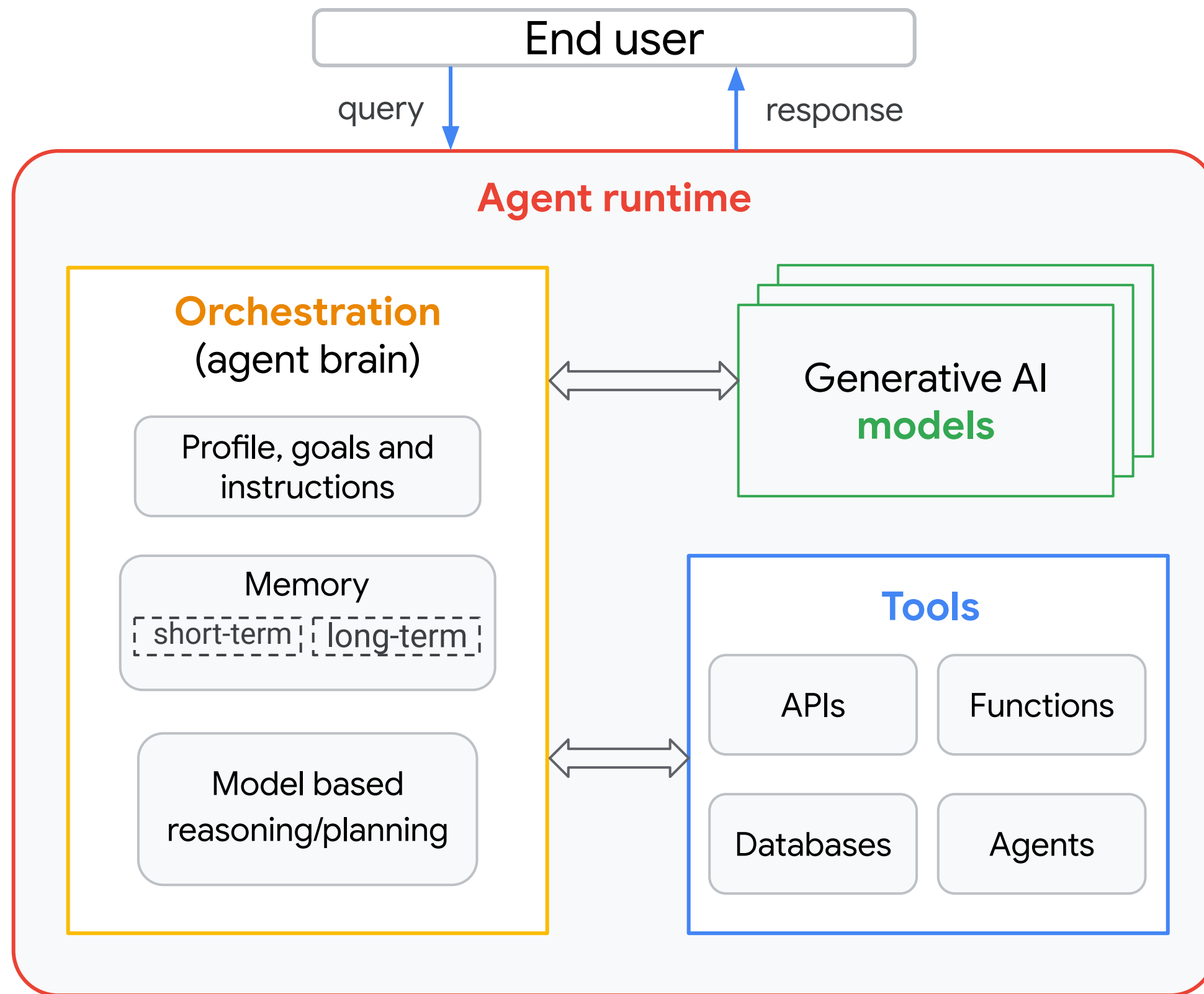


ADK key concepts

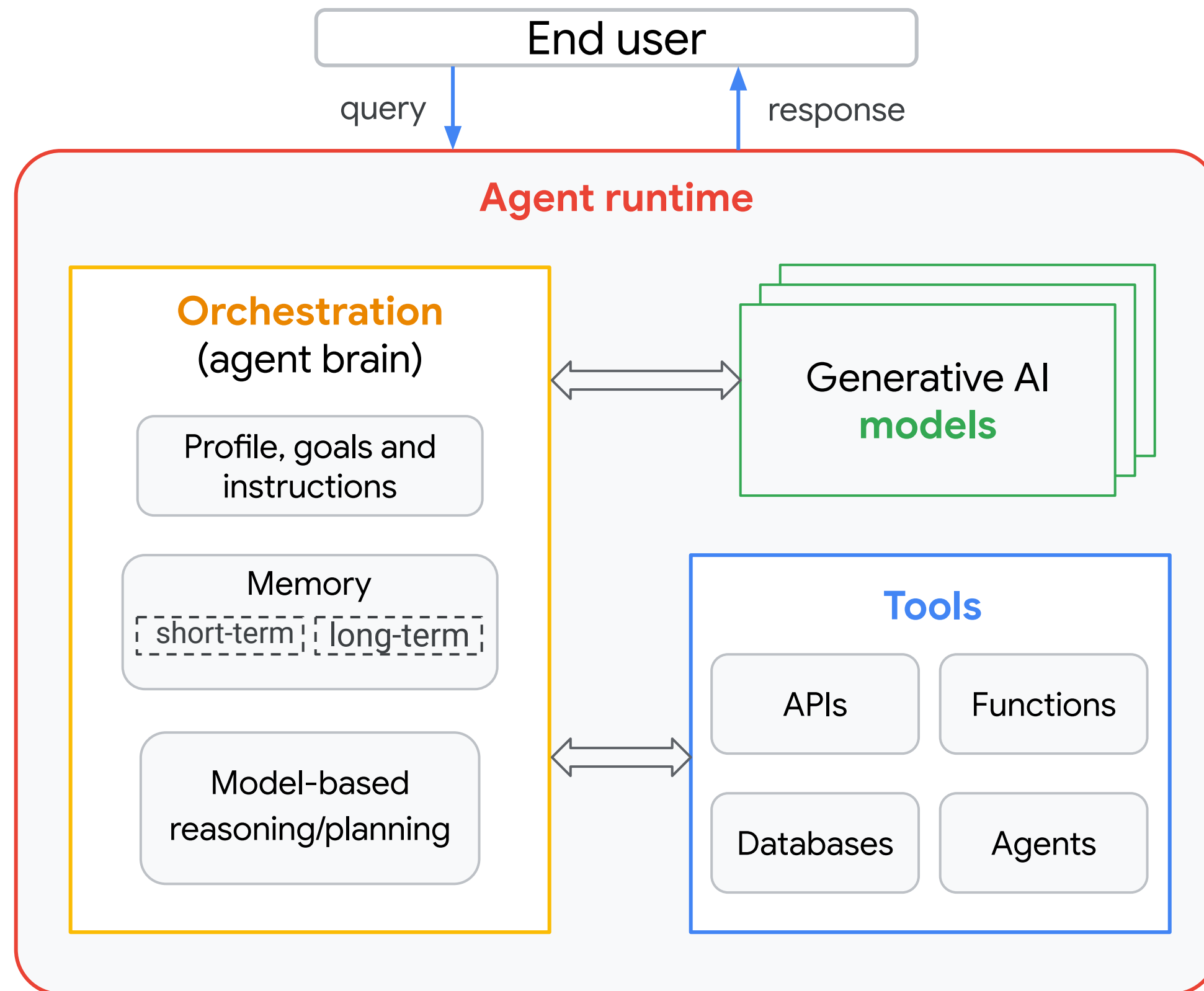


Agent Development Kit SDK

Key components



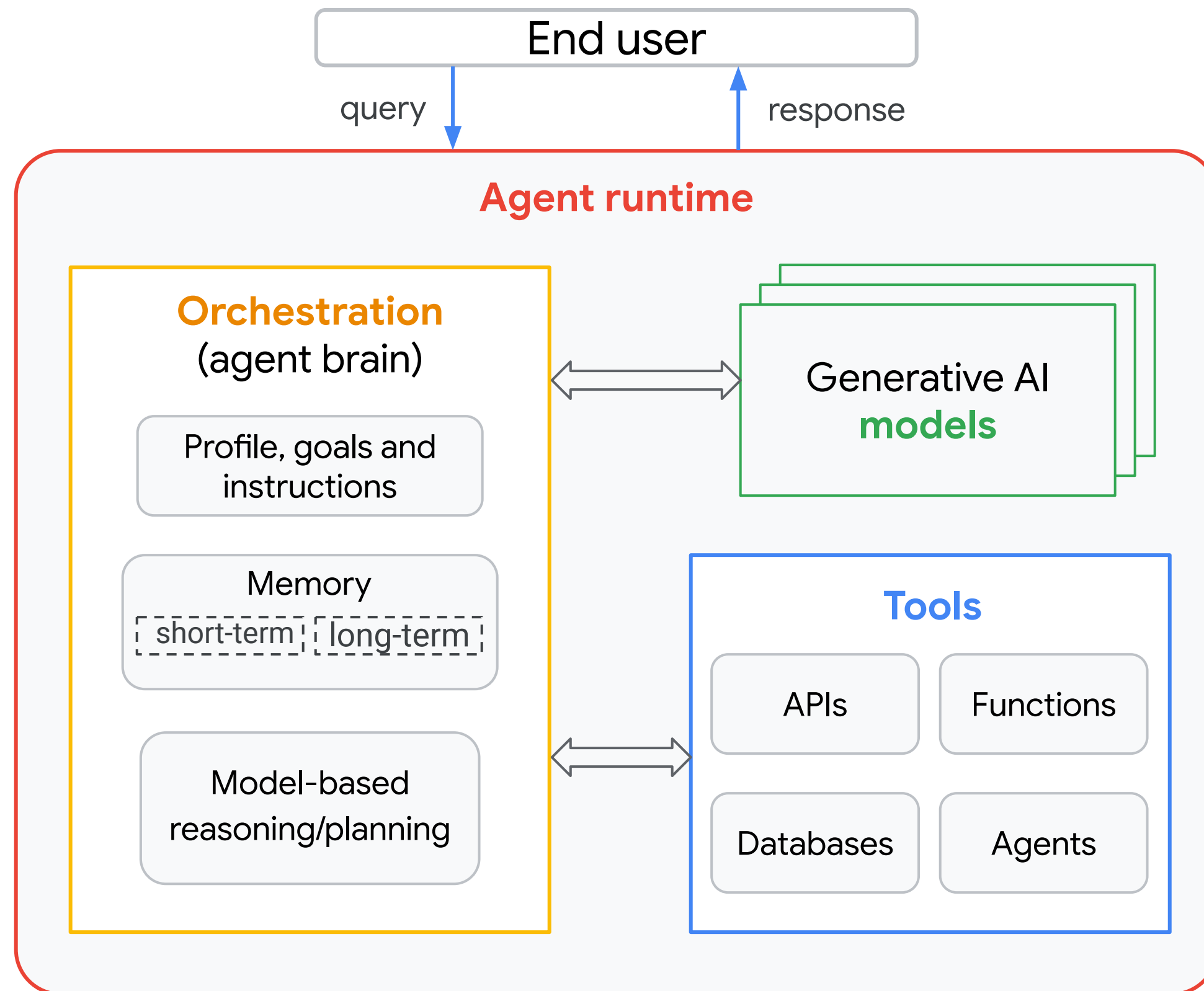
Agent Development Kit SDK



Key components

- **Models**: Used to reason over goals, determine the plan and generate a response

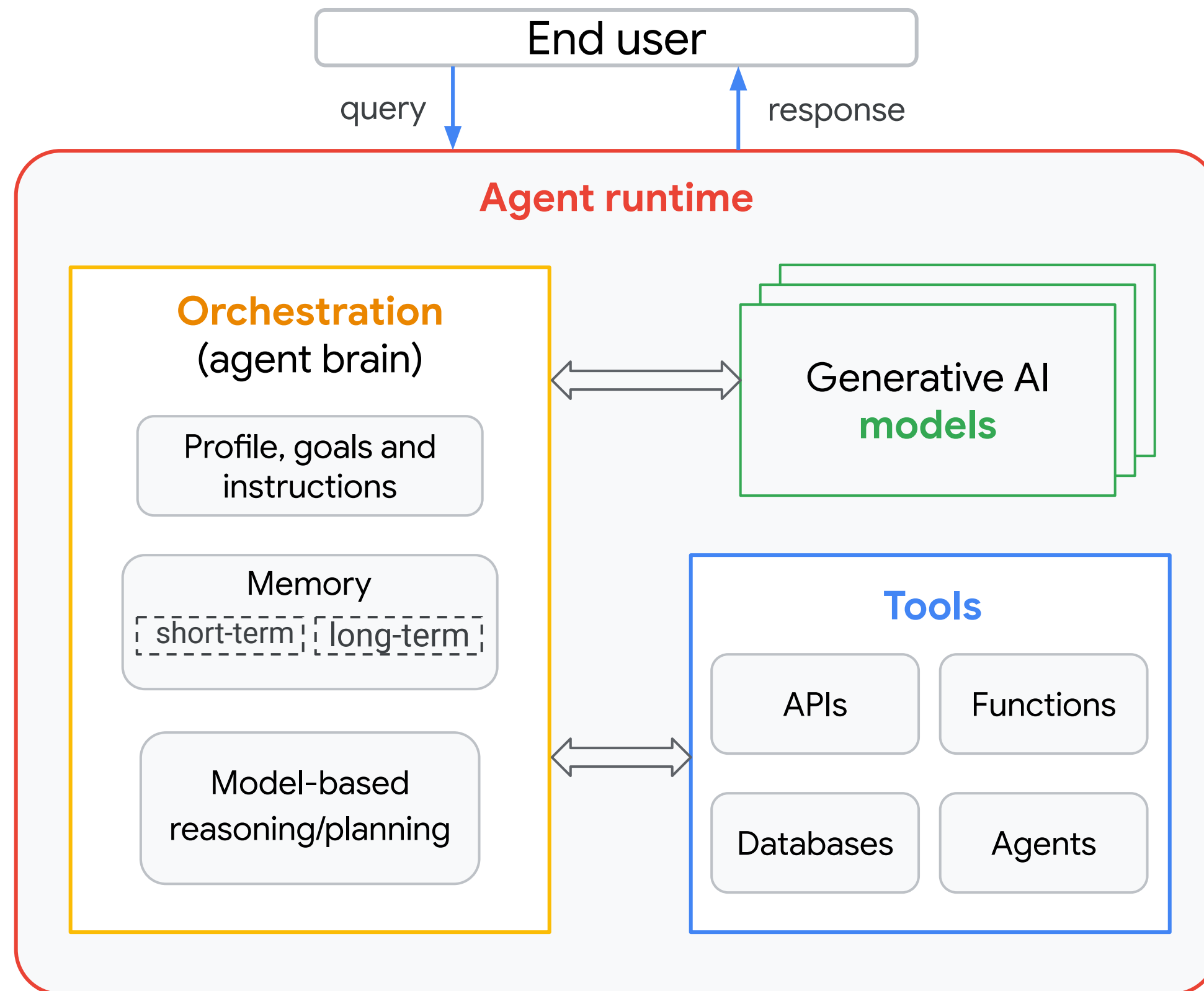
Agent Development Kit SDK



Key components

- **Models**: Used to reason over goals, determine the plan and generate a response
- **Tools**: Fetch data, perform actions or transactions by calling other APIs or services

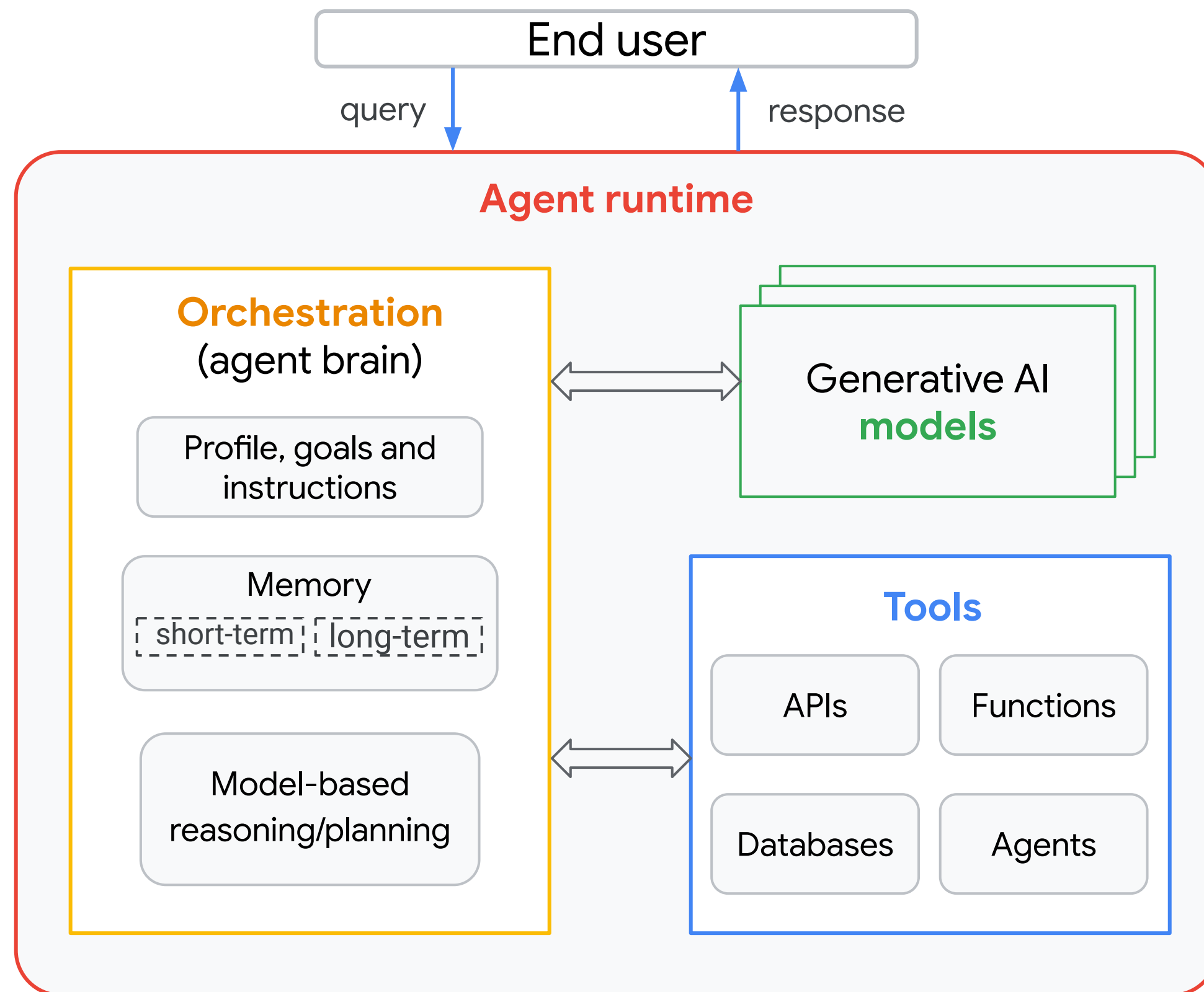
Agent Development Kit SDK



Key components

- **Models:** Used to reason over goals, determine the plan and generate a response
- **Tools:** Fetch data, perform actions or transactions by calling other APIs or services
- **Orchestration:** Maintain memory and state (including the approach used to plan), tools, data provided and fetched

Agent Development Kit SDK



Key components

- **Models:** Used to reason over goals, determine the plan and generate a response
- **Tools:** Fetch data, perform actions or transactions by calling other APIs or services
- **Orchestration:** Maintain memory and state (including the approach used to plan), tools, data provided and fetched
- **Runtime:** Execute the system when invoked

Configure ADK

Configure ADK

- ✓ Set up ADK environment
- ✓ Install ADK
- ✓ Configure the model
- ✓ Explore ways to run agents



Create a virtual environment, and use it to install ADK

- Open your terminal
- Navigate to the directory to create your project e.g. ~/projects
- Run this command to create a virtual environment named .adk

```
python3 -m venv .adk
```

This is highly recommended to isolate your Agent Development Kit project's dependencies, and prevent conflicts with other Python projects.

Activate the virtual environment by running the command for Linux and MacOS, or for Windows

Linux and MacOS:

```
source .adk/bin/activate
```

Windows:

```
.adk\Scripts\activate
```

Your terminal prompt should now change to indicate that the virtual environment is active
e.g. (.adk) yourname@yourcomputer:~\$

Deactivate the virtual environment

Run

```
deactivate
```

With your virtual environment activated, install ADK using pip

Install ADK:

```
pip install google-adk # Python 3.9+
```

Verify Installation (optional):

```
pip show google-adk
```

or

```
pip list | grep google-adk
```

ADK project structure and example

ADK directory structure

- ✓ There is a directory structure that must be maintained to organize your agents
- ✓ `__init__` file with an import from agent
- ✓ `agent.py`

```
✓ agent_grammar
  🐍 __init__.py
  🐍 agent.py
✓ agent_maths
  🐍 __init__.py
  🐍 agent.py
```

Use of .env files



Helps provide configurations for each agent



Authorize through the Gemini API or Vertex AI projects

```
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=very-best-project
GOOGLE_CLOUD_LOCATION=us-central1
MODEL='gemini-2.0-flash-exp'
```

Basic agent code

```
# Create a basic agent with instructions and greeting only
root_agent = Agent(
    model=MODEL,
    name="basic_agent",
    description=" " " "
        This agent responds to inquiries about its creation by stating it was built
        using the Google Agent Framework.
    " " " ,
    instruction=" " " "
        If they ask you how you were created, tell them you were created with the Google
        Agent Framework.
    " " " ,
    generate_content_config=types.GenerateContentConfig(temperature=0.2)
)
```

Root agent variable

Basic agent code

```
# Create a basic agent with instructions and greeting only
root_agent = Agent(
    model=MODEL,
    name="basic_agent",
    description=" " " "
        This agent responds to inquiries about its creation by stating it was built
        using the Google Agent Framework.
    " " " ,
    instruction=" " " "
        If they ask you how you were created, tell them you were created with the Google
        Agent Framework.
    " " " ,
    generate_content_config=types.GenerateContentConfig(temperature=0.2)
)
```

Description

Basic agent code

```
# Create a basic agent with instructions and greeting only
root_agent = Agent(
    model=MODEL,
    name="basic_agent",
    description=" " " "
        This agent responds to inquiries about its creation by stating it was built
        using the Google Agent Framework.
    " " " ,
    instruction=" " " "
        If they ask you how you were created, tell them you were created with the Google
        Agent Framework.
    " " " ,
    generate_content_config=types.GenerateContentConfig(temperature=0.2)
)
```

Instruction

Basic agent code

```
# Create a basic agent with instructions and greeting only
root_agent = Agent(
    model=MODEL,
    name="basic_agent",
    description=" " " "
        This agent responds to inquiries about its creation by stating it was built
        using the Google Agent Framework.
    " " "
    ,
    instruction=" " " "
        If they ask you how you were created, tell them you were created with the Google
        Agent Framework.
    " " "
    ,
    generate_content_config=types.GenerateContentConfig(temperat
)

```

[generate_content_config](#)

Open Google Cloud Skills Boost

- 01 Open an incognito window (or private/anonymous window).
- 02 Go to the Qwiklabs URL cloudskillsboost.google.
- 03 Sign in with existing account, or join with new account, using the **email you used to register** for the course.
- 04 Launch the course from **In Progress** section of your home page.

Jumpstart your cloud career

Not sure where to start? Find featured learning below. We give you temporary credentials to actual cloud resources, so you can learn the cloud using the real thing.

In Progress



Access issues

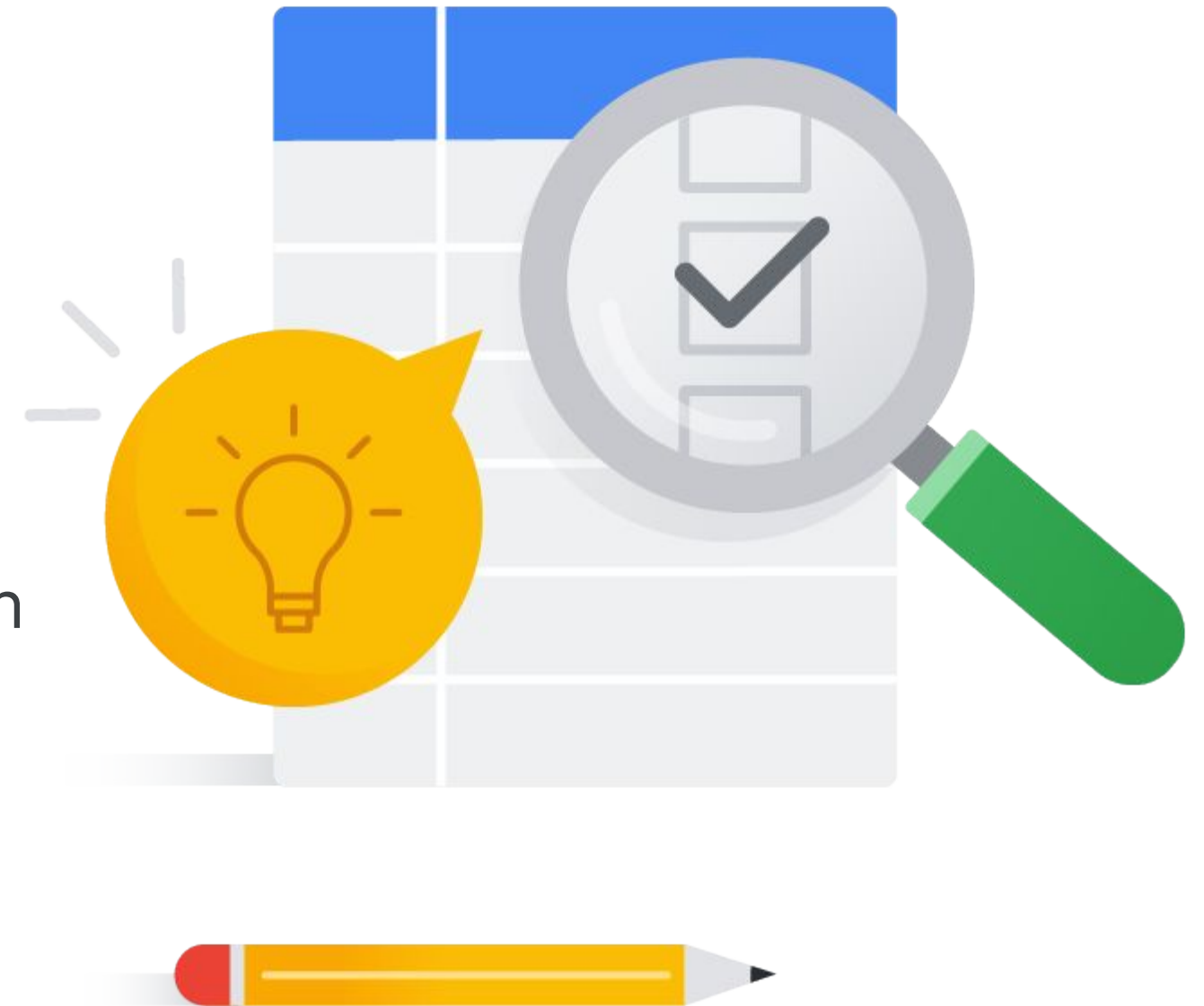
If you encounter any problems while setting up your Qwiklabs account, follow the troubleshooting steps here: <http://traininghelp.cloudlearning.io>

Lab: Get started with Agent Development Kit (ADK)

🕒 45 min

Overview

In this lab, you will create a single agent that can use a search tool. You will test agents in ADK's browser UI, from a CLI chat interface, and programmatically from within a script.



Interacting with agents

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

The way you define your agent is the same regardless of how you choose to interact with it.
The difference lies in [how you initiate and manage the interaction](#).

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

- Interact with your agent through a user-friendly web browser
- Use for visual interaction and monitoring agent behavior
- Only use for local testing - not suitable for a production environment

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

Get started with Web UI

1. Open your terminal
2. Use `cd` to navigate to the directory
3. Run this command from your project folder to start a local web server and user interface.

```
adk web
```

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

- Use terminal commands to interact directly with your agent
- Use for quick tasks, scripting, automation, and developers comfortable with terminal commands
- Only use for local testing - not suitable for a production environment

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

Get started with CLI

1. Open your terminal
2. Use `cd` to navigate to the directory
3. Run this command to start the agent

```
adk run my_google_search_agent # Replace with your agent's folder name
```

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

- Run your agent as a REST API, allowing other applications to communicate with it
- Use for integration with other applications, building services that use the agent, and remote access to the agent
- Use for production environments

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

Get started with API server

1. Open your terminal
2. Use `cd` to navigate to the directory
3. Run this command to start a local API server

```
adk api_server my_google_search_agent # Replace with your agent's folder name
```

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

- Integrate ADK directly into your Python applications, or interactive notebooks e.g. Jupyter, Colab
- Use a Session and Runner, and define and interact with your agent within the same file or notebook cell
- Provides deep integration within applications, custom workflows, notebooks, and fine-grained control over agent execution
- Use for production environments

Ways to interact with your agents

Web UI

Command Line
Interface

API server

Programmatic
Interface

Get started with Programmatic Interface

1. Set up Memory.
 - `InMemorySessionService`
 - `InMemoryArtifactService`
2. Create a new session.
3. Prepare content for the agent.
4. Use a Runner to execute the agent's logic.
5. Process the event stream to get the final response.

Invoking ADK agents programmatically

```
root_agent = LLMAgent(...)

runner = InMemoryRunner(agent=root_agent)

session = runner.session_service.create_session(
    app_name=runner.app_name, user_id="test_user"
)

user_input = "My prompt"
content = UserContent(parts=[Part(text=user_input)])
for event in runner.run(
    user_id=session.user_id, session_id=session.id,
    new_message=content
):
    for part in event.content.parts:
        print(part.text)
```

Create your agent

Create a runner

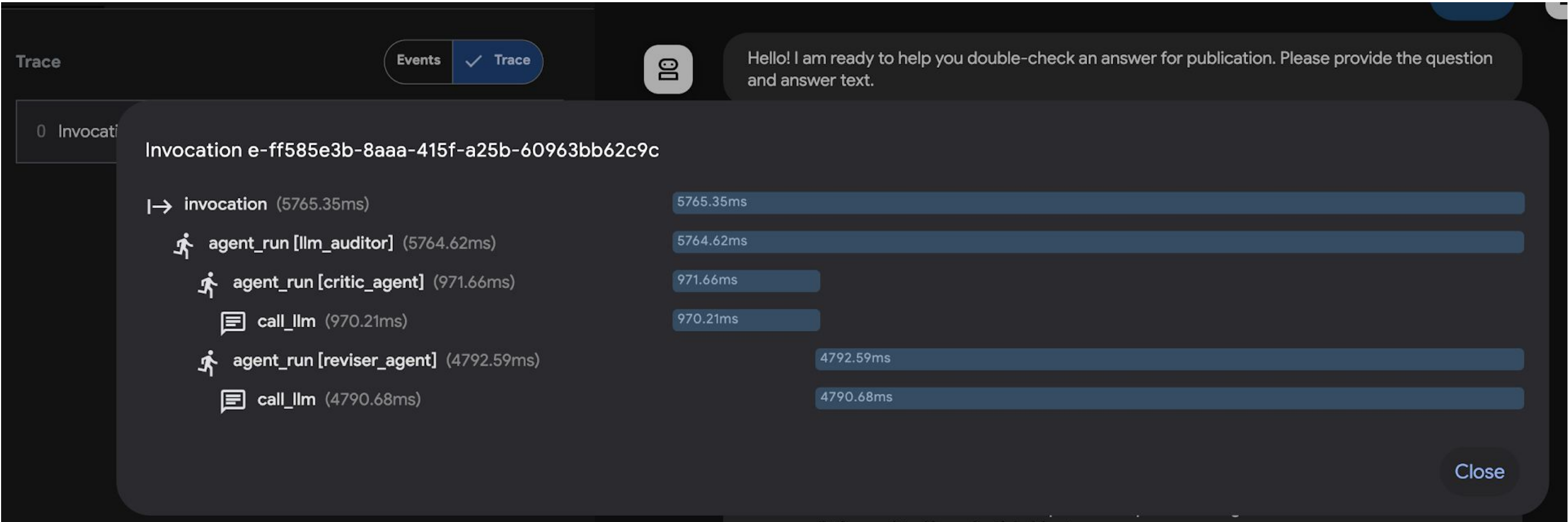
Create a session

Content prepared

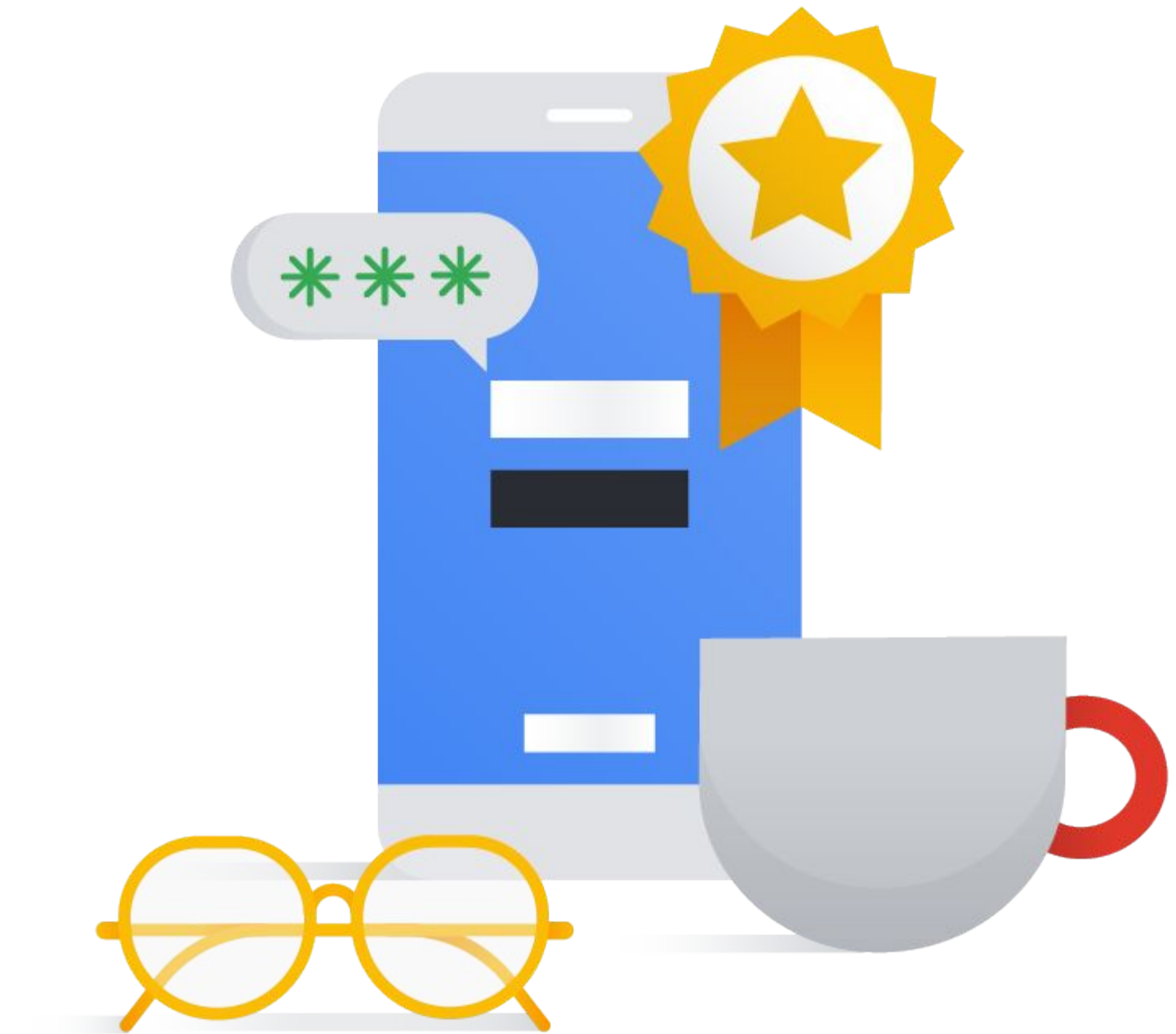
Run and iterate
through response

Trace agent runs

In the Dev UI, you can follow the overall and individual spans of each step of your agents' invocations.



Now let's do a short
quiz to **check your
knowledge!**



Quiz | Question 01

Question

What does BiDi streaming support in ADK enable?

- A. Static text display
- B. Real-time, interactive experiences
- C. One-way data transfer only
- D. Offline data storage

Quiz | Question 01

Answer

What does BiDi streaming support in ADK enable?

- A. Static text display
- B. Real-time, interactive experiences
- C. One-way data transfer only
- D. Offline data storage



Quiz | Question 02

Question

What is the purpose of Artifact Management in ADK?

- A. To archive old files
- B. To create music files
- C. To organize emails
- D. To manage access to files

Quiz | Question 02

Answer

What is the purpose of Artifact Management in ADK?

- A. To archive old files
- B. To create music files
- C. To organize emails
- D. To manage access to files



Quiz | Question 03

Question

What does a Session include in ADK?

- A. A user password
- B. The history of a single conversation
- C. Information on the client device
- D. The history and state of a single conversation

Quiz | Question 03

Answer

What does a Session include in ADK?

- A. A user password
- B. The history of a single conversation
- C. Information on the client device
- D. The history and state of a single conversation



Quiz | Question 04

Question

What is the role of the Runner in ADK?

- A. To write documentation
- B. To manage the execution flow of agents
- C. To design user interfaces
- D. To store server backups

Quiz | Question 04

Answer

What is the role of the Runner in ADK?

- A. To write documentation
- B. To manage the execution flow of agents
- C. To design user interfaces
- D. To store server backups

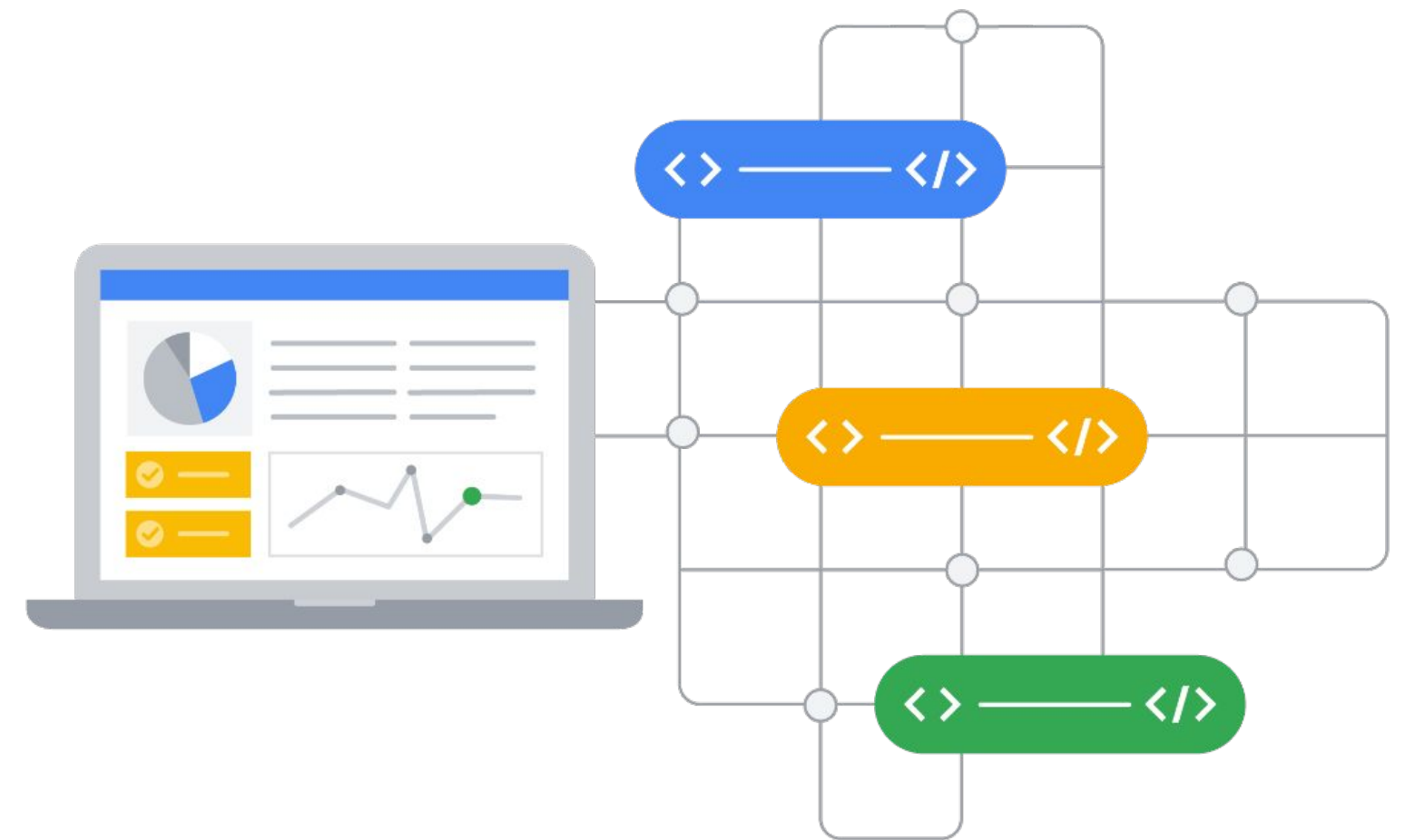


Callbacks



Callbacks

- Callbacks allow you to interrupt the agent's execution lifecycle to observe, modify, or augment the agent's operations directly.
- ADK provides different types of callbacks that trigger at various stages of an agent's execution - defined on the Base Agent and the more specialized LLM Agent classes.
- To use callbacks effectively, understand when each callback fires and what context it receives.



Agent lifecycle callbacks

`before_agent_callback`

`after_agent_callback`

Agent lifecycle callbacks

before_agent_callback

- Setting up resources or state needed only for this specific agent's run
- Performing validation checks on the session state (`callback_context.state`) before execution
- Logging the entry point of the agent's activity
- Modifying session state before the agent uses it
- Returning content to skip the agent's execution

after_agent_callback

Agent lifecycle callbacks

before_agent_callback

- Setting up resources or state needed only for this specific agent's run
- Performing validation checks on the session state (`callback_context.state`) before execution
- Logging the entry point of the agent's activity
- Modifying session state before the agent uses it
- Returning content to skip the agent's execution

after_agent_callback

- Cleanup tasks
- Post-execution validation
- Logging the completion of an agent's activity
- Modifying final state
- Augmenting/replacing the agent's final output

LLM interaction callbacks

`before_model_callback`

`after_model_callback`

LLM interaction callbacks

before_model_callback

- Inspection and modification of the request going to the LLM
 - Adding dynamic instructions
 - Injecting few-shot examples based on state
 - Modifying model config.
 - Implementing guardrails (like profanity filters)
 - Implementing request-level caching

after_model_callback

LLM interaction callbacks

before_model_callback

- Inspection and modification of the request going to the LLM
 - Adding dynamic instructions
 - Injecting few-shot examples based on state
 - Modifying model config.
 - Implementing guardrails (like profanity filters)
 - Implementing request-level caching

after_model_callback

- Inspection or modification of the raw LLM response
 - Logging model outputs
 - Reformatting responses
 - Censoring sensitive information generated by the model
 - Parsing structured data from the LLM response and storing it
 - Handling specific error codes

Tool execution callbacks

`before_tool_callback`

`after_tool_callback`

Tool execution callbacks

before_tool_callback

- Inspection and modification of tool arguments
- Performing authorization checks before execution
- Logging tool usage attempts
- Implementing tool-level caching

after_tool_callback

Tool execution callbacks

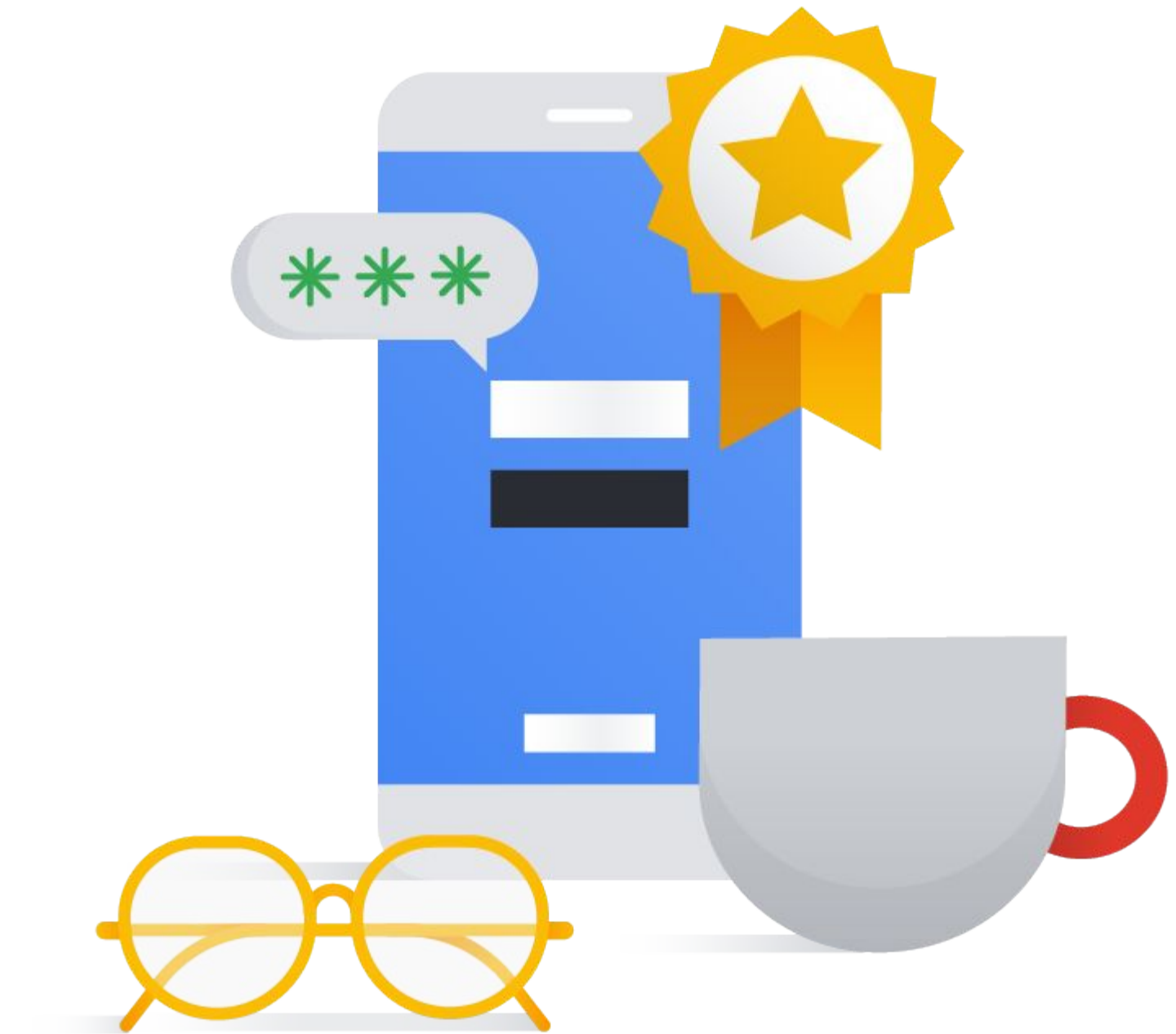
before_tool_callback

- Inspection and modification of tool arguments
- Performing authorization checks before execution
- Logging tool usage attempts
- Implementing tool-level caching

after_tool_callback

- Inspection and modification of the tool's result before it's sent back to the LLM (potentially after summarization)
 - Logging tool results
 - Post-processing or formatting results
 - Saving specific parts of the result to the session state

Now let's do a short
quiz to **check your
knowledge!**



Quiz | Question 01

Question

What is the purpose of the 'before_agent_callback'?

- A. Cleanup tasks before tool execution
- B. Modifying final state after execution
- C. Setting up resources before agent execution
- D. Logging the completion of an agent's activity

Quiz | Question 01

Answer

What is the purpose of the 'before_agent_callback'?

- A. Cleanup tasks before tool execution
- B. Modifying final state after execution
- C. Setting up resources before agent execution
- D. Logging the completion of an agent's activity



Quiz | Question 02

Question

What do model interaction callbacks provide hooks around?

- A. The interaction with the Large Language Model
- B. The agent's overall lifecycle
- C. The execution of tools
- D. The execution of Workflow Agents

Quiz | Question 02

Answer

What do model interaction callbacks provide hooks around?

- A. The interaction with the Large Language Model
- B. The agent's overall lifecycle
- C. The execution of tools
- D. The execution of Workflow Agents

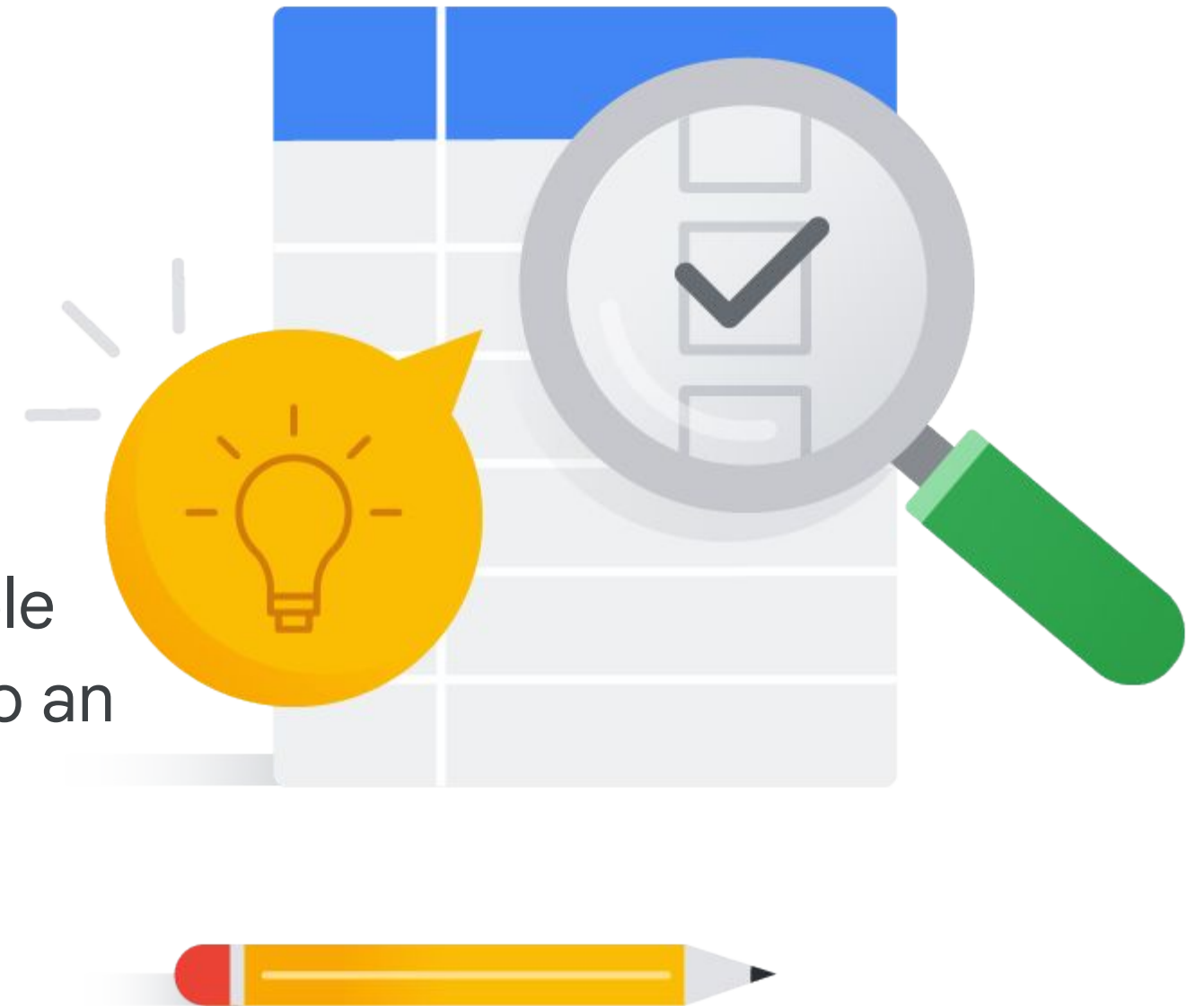


Lab: Empower ADK agents with tools

🕒 45 min

Overview

In this lab, you will learn about the ecosystem of tools available to ADK agents. You will also learn how to provide a function to an agent as a custom tool.

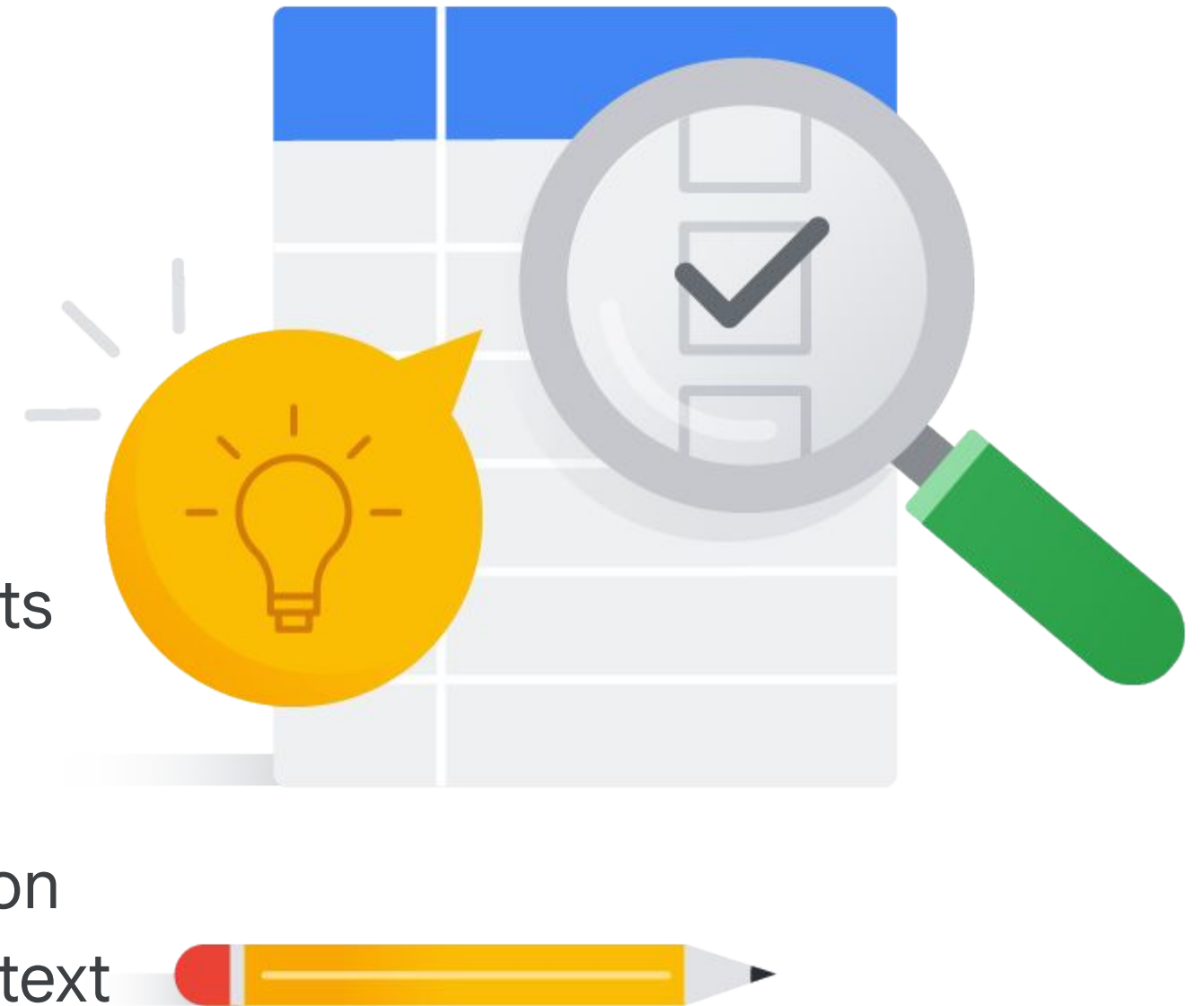


Optional Lab: Use Model Context Protocol (MCP) Tools with ADK Agents

🕒 45 min

Overview

In this lab, you will learn how to use ADK agents as MCP clients and expose tools through a custom MCP server. You will configure the server to allow standardized communication between agents and tools. This lab shows seamless integration between LLMs and external capabilities using the Model Context Protocol.



Google Cloud